

SQL Assignment 3

1 Completed Sales Orders (Physical Items)

Business Problem:

Merchants need to track only physical items (requiring shipping and fulfillment) for logistics and shipping-cost analysis.

```
SELECT oh.order_id,  
       oi.order_item_seq_id,  
       p.product_id,  
       p.product_type_id,  
       oh.sales_channel_enum_id,  
       oh.order_date,  
       oh.entry_date,  
       os.status_id,  
       os.status_datetime,  
       oh.order_type_id,  
       oh.product_store_id  
  
FROM ORDER_HEADER oh  
JOIN ORDER_ITEM oi  
ON oh.order_id = oi.order_id  
JOIN ORDER_STATUS os  
ON oh.order_id = os.order_id AND oi.order_item_seq_id = os.order_item_seq_id  
JOIN PRODUCT p  
ON oi.product_id = p.product_id  
  
WHERE p.is_variant = 'Y' AND p.is_virtual = 'N'  
      AND oh.order_type_id = 'SALES_ORDER'  
      AND os.status_id = 'ITEM_COMPLETED';;
```

```
FROM return_header rh
JOIN return_item ri
```

ON rh.return_id = rs.return_id

```
SELECT rh.return_id,
       ri.order_id,
       oh.product_store_id,
       rs.status_datetime,
       oh.order_name,
       rh.from_party_id,
       rh.return_date,
       rh.entry_date,
       rh.return_channel_enum_id

FROM return_header rh
JOIN return_item ri
ON rh.return_id = ri.return_id
JOIN order_header oh
ON ri.order_id = oh.order_id
JOIN return_status rs
ON rh.return_id = rs.return_id

WHERE rs.status_id = 'RETURN_COMPLETED';
```

00:00:00.27

Filter results

SELECT rh.return_id, ri.order_id, oh.product_store_id, rs.status_datetime, oh.order_name, rh.from_party_id, rh.return_date, rh.entry_date, rh.return_channel_enum_id FROM return_header rh JOIN return_item ri ON rh.return_id = ri.return_id									
return_id	order_id	product_store_id	status_datetime	order_name	from_party_id	return_date	entry_date	return_channel_enum_id	
M100000	M100017	STORE	2026-05-02T07:07:33.894000	#HCD1018	M100007	2026-01-17T04:27:16	2026-05-02T07:07:33.891000	ECOM_RTN_CHANNEL	
M100000	M100017	STORE	2026-05-02T07:07:33.895000	#HCD1018	M100007	2026-01-17T04:27:16	2026-05-02T07:07:33.891000	ECOM_RTN_CHANNEL	
M100001	M100018	STORE	2026-05-02T07:07:35.316000	#HCD1019	M100007	2026-01-17T04:47:06	2026-05-02T07:07:35.315000	ECOM_RTN_CHANNEL	
M100001	M100018	STORE	2026-05-02T07:07:35.316000	#HCD1019	M100007	2026-01-17T04:47:06	2026-05-02T07:07:35.315000	ECOM_RTN_CHANNEL	
M100002	M100019	STORE	2026-05-02T07:07:35.386000	#HCD1020	M100007	2026-01-17T05:00:00	2026-05-02T07:07:35.384000	ECOM_RTN_CHANNEL	
M100002	M100019	STORE	2026-05-02T07:07:35.386000	#HCD1020	M100007	2026-01-17T05:00:00	2026-05-02T07:07:35.384000	ECOM_RTN_CHANNEL	

The merchandising team needs a list of orders that only have one return.

```
ON rh.return_id = ri.return_id
```

JOIN order_header oh
ON ri.order_id = oh.order_id

WHERE oh.order_id IN (
SELECT order_id from return_item
GROUP BY order_id
HAVING COUNT(*) = 1
)

GROUP BY p.party_id, pr.first_name
ORDER BY pr.first_name;

```
1  SELECT p.party_id,  
2         pr.first_name  
3  
4  FROM party p  
5  JOIN person pr  
6  ON p.party_id = pr.party_id  
7  JOIN return_header rh  
8  ON p.party_id = rh.from_party_id  
9  JOIN return_item ri  
10 ON rh.return_id = ri.return_id  
11 JOIN order_header oh  
12 ON ri.order_id = oh.order_id  
13  
14 WHERE oh.order_id IN (  
15     SELECT order_id from return_item  
16     GROUP BY order_id  
17     HAVING COUNT(*) = 1  
18 )  
19  
20 GROUP BY p.party_id, pr.first_name  
21 ORDER BY pr.first_name;
```

RUN LIMIT: 1000 00:00:00.23

RESULTS QUERY HISTORY

CREATE CHART DOWNLOAD TO CSV COPY TO CL

SELECT p.party_id, pr.first_name FROM party p JOIN person

party_id	first_name
M100007	Shrikuti

4 Returns and Appeasements

Business Problem:

The retailer needs the total amount of items, were returned as well as how many appeasements were issued.

```
SELECT
    r.total_returns,
    r.return_dollar_total,
    a.total_appeasements,
    a.appeasement_dollar_total
FROM
    (
        SELECT
            COUNT(DISTINCT return_id) AS total_returns,
            SUM(amount) AS return_dollar_total
        FROM return_item_billing
    ) r,
    (
        SELECT
            COUNT(*) AS total_appeasements,
            SUM(amount) AS appeasement_dollar_total
        FROM return_adjustment
        WHERE return_adjustment_type_id = 'APPEASEMENT'
    ) a;
```

5 Detailed Return Information

Business Problem:

Certain teams need granular return data (reason, date, refund amount) for analyzing return rates, identifying recurring issues, or updating policies.

```
SELECT rh.return_id,
    rh.entry_date,
    ra.return_adjustment_type_id,
    ri.return_reason_id,
    rib.amount,
```

```
ra.comments,  
oh.order_id,  
oh.order_date,  
rh.entry_date AS return_date,  
oh.product_store_id
```

```
FROM return_header rh  
JOIN return_item ri  
  ON rh.return_id = ri.return_id  
JOIN return_item_billing rib  
  ON ri.return_id = rib.return_id  
AND ri.return_item_seq_id = rib.return_item_seq_id  
JOIN order_header oh  
  ON ri.order_id = oh.order_id  
LEFT JOIN return_adjustment ra  
  ON rh.return_id = ra.return_id;
```

6 Orders with Multiple Returns

Business Problem:

Analyzing orders with multiple returns can identify potential fraud, chronic issues with certain items, or inconsistent shipping processes

```
SELECT  
  ri.order_id,  
  rh.return_id,  
  rh.return_date,  
  ri.return_reason_id AS return_reason,  
  ri.return_quantity AS return_quantity  
FROM return_header rh  
JOIN return_item ri  
  ON rh.return_id = ri.return_id  
WHERE ri.order_id IN (  
  SELECT order_id  
  FROM return_item  
  GROUP BY order_id  
  HAVING COUNT(DISTINCT return_id) > 1  
);
```

7 Store with Most One-Day Shipped Orders (Last Month)

Business Problem:

Identify which facility (store) handled the highest volume of “one-day shipping” orders in the previous month, useful for operational benchmarking.

```
SELECT f.FACILITY_ID,
       f.FACILITY_NAME,
       COUNT(DISTINCT oisg.order_id) AS TOTAL_ONE_DAY_SHIP_ORDERS,
       DATE_FORMAT(DATE_SUB(CURRENT_DATE,INTERVAL 1 MONTH),'%Y-%m') AS
REPORTING_PERIOD

FROM ORDER_ITEM_SHIP_GROUP oisg
JOIN FACILITY f
ON oisg.FACILITY_ID = f.FACILITY_ID
JOIN ORDER_HEADER oh
ON oisg.ORDER_ID = oh.ORDER_ID WHERE oisg.SHIPMENT_METHOD_TYPE_ID =
'NEXT_DAY'
AND oh.ORDER_DATE >= DATE_FORMAT(DATE_SUB(CURRENT_DATE, INTERVAL 1
MONTH),'%Y-%m-01') AND oh.ORDER_DATE < DATE_FORMAT(CURRENT_DATE,
'%Y-%m-01')
AND oh.STATUS_ID = 'ORDER_COMPLETED'
GROUP BY f.FACILITY_ID, f.FACILITY_NAME
ORDER BY TOTAL_ONE_DAY_SHIP_ORDERS DESC
LIMIT 1;
```

```
T f.FACILITY_ID,
f.FACILITY_NAME,
COUNT(DISTINCT oisg.order_id) AS TOTAL_ONE_DAY_SHIP_ORDERS,
DATE_FORMAT(DATE_SUB(CURRENT_DATE,INTERVAL 1 MONTH),'%Y-%m') AS REPORTING_PERIOD

FROM ORDER_ITEM_SHIP_GROUP oisg
JOIN FACILITY f
ON oisg.FACILITY_ID = f.FACILITY_ID
JOIN ORDER_HEADER oh
```

N

LIMIT: 1000

00:00:00.39

SAVE



COPY LINK



QUERY HISTORY

HART

DOWNLOAD TO CSV

COPY TO CLIPBOARD

Filter results

FACILITY_ID, f.FACILITY_NAME, COUNT(DISTINCT oisg.order_id) AS TOTAL_ONE_DAY_SHIP_ORDERS, DATE_FORMAT(DATE_SUB(CURRENT_DATE,INT...

1 row

	FACILITY_NAME	TOTAL_ONE_DAY_SHIP_ORDERS	REPORTING_PERIOD
	Austin	5	2026-05

8 List of Warehouse Pickers

Business Problem:

Warehouse managers need a list of employees responsible for picking and packing orders to manage shifts, productivity, and training needs.

```
SELECT fp.PARTY_ID,  
       CONCAT(p.FIRST_NAME, ' ', p.LAST_NAME) AS NAME,  
       fp.ROLE_TYPE_ID,  
       fp.FACILITY_ID,  
       pty.STATUS_ID AS STATUS  
  
FROM FACILITY_PARTY fp  
JOIN PERSON p  
ON fp.PARTY_ID = p.PARTY_ID  
JOIN PARTY pty  
ON fp.PARTY_ID = pty.PARTY_ID WHERE fp.ROLE_TYPE_ID = '%PICKER%'  
AND (  
fp.THRU_DATE IS NULL OR fp.THRU_DATE > CURRENT_TIMESTAMP  
)  
  
ORDER BY fp.FACILITY_ID , fp.ROLE_TYPE_ID, NAME;
```

9 Total Facilities That Sell the Product

Business Problem:

Retailers want to see how many (and which) facilities (stores, warehouses, virtual sites) currently offer a product for sale.

```
SELECT p.PRODUCT_ID,  
       p.PRODUCT_NAME,  
       COUNT(DISTINCT pf.FACILITY_ID) AS FACILITY_COUNT,  
       GROUP_CONCAT(  
DISTINCT pf.FACILITY_ID
```

```

ORDER BY pf.FACILITY_ID SEPARATOR ', ') AS 'list of FACILITY_IDs'
FROM PRODUCT p
JOIN PRODUCT_FACILITY pf
ON p.PRODUCT_ID = pf.PRODUCT_ID
WHERE (p.SALES_DISCONTINUATION_DATE IS NULL OR
p.SALES_DISCONTINUATION_DATE > CURRENT_TIMESTAMP)

GROUP BY p.PRODUCT_ID, p.INTERNAL_NAME
ORDER BY FACILITY_COUNT DESC;

```

RESULTS

QUERY HISTORY

CREATE CHART

DOWNLOAD TO CSV

COPY TO CLIPBOARD

Filter results

SELECT p.PRODUCT_ID, p.PRODUCT_NAME, COUNT(DISTINCT pf.FACILITY_ID) AS FACILITY_COUNT, GROUP_CONCAT(DISTINCT pf.FACILITY_ID ORDER BY pf.FACILITY_ID SEPARATOR ', ') AS 'list of FACILITY_IDs'

57 rows

PRODUCT_ID	PRODUCT_NAME	FACILITY_COUNT	list of FACILITY_IDs
M102454	XL / Yellow	6	Austin, DEMO_FACILITY, MAPLEWOOD, SV_STORE, TEST_FACILITY, Warehouse
M102871	32 / Blue	6	Austin, DEMO_FACILITY, HARSH_RETAIL_STORE, MAPLEWOOD, Toronto, Warehouse
M103633	XS / Blue	6	Austin, DEMO_FACILITY, MAPLEWOOD, Scottsdale, Toronto, Warehouse
M103588	XS / Blue	4	Austin, DEMO_FACILITY, MAPLEWOOD, Warehouse
M103006	32 / Blue	4	Austin, DEMO_FACILITY, HARSH_RETAIL_STORE, MAPLEWOOD
M102649	XS / Blue	3	DEMO_FACILITY, MAPLEWOOD, Scottsdale
10545	Baked Balance-n-Brighten Color Correcting Foundation - Porcelain	3	Austin, DEMO_FACILITY, MAPLEWOOD
M103145	M / Blue	3	Austin, MAPLEWOOD, Toronto
M103289	XS / Brown	2	Austin, MAPLEWOOD
M102912	34 / Black	2	Austin, MAPLEWOOD

10 Total Items in Various Virtual Facilities

Business Problem:

Retailers need to study the relation of inventory levels of products to the type of facility it's stored at. Retrieve all inventory levels for products at locations and include the facility type Id. Do not retrieve facilities that are of type Virtual.

```

SELECT
    ii.PRODUCT_ID,

```

```

ii.FACILITY_ID,
f.FACILITY_TYPE_ID,
SUM(ii.QUANTITY_ON_HAND_TOTAL) AS QOH,
SUM(ii.AVAILABLE_TO_PROMISE_TOTAL) AS ATP
FROM
INVENTORY_ITEM ii
JOIN
FACILITY f ON ii.FACILITY_ID = f.FACILITY_ID
WHERE
f.FACILITY_TYPE_ID NOT IN (
    SELECT FACILITY_TYPE_ID
    FROM FACILITY_TYPE
    WHERE FACILITY_TYPE_ID = 'VIRTUAL_FACILITY'
    OR PARENT_TYPE_ID = 'VIRTUAL_FACILITY'
)
GROUP BY
ii.PRODUCT_ID,
ii.FACILITY_ID,
f.FACILITY_TYPE_ID;

```

RESULTS QUERY HISTORY						
		CREATE CHART DOWNLOAD TO CSV COPY TO CLIPBOARD Filter results				
		SELECT ii.PRODUCT_ID, ii.FACILITY_ID, f.FACILITY_TYPE_ID, SUM(ii.QUANTITY_ON_HAND_TOTAL) AS QOH, SUM(ii.AVAILABLE_TO_PROMISE_TOTAL) AS ATP FROM INVENTORY_ITEM ii JOIN FACILITY f ON ii.FACIL_			99 rows	
PRODUCT_ID	FACILITY_ID	FACILITY_TYPE_ID	QOH	ATP		
M102454	DEMO_FACILITY	RETAIL_STORE	74	74		
M102454	TEST_FACILITY	RETAIL_STORE	50	50		
M102454	Austin	RETAIL_STORE	-14	-14		
M103993	Austin	RETAIL_STORE	-1	-5		
M103837	DEMO_FACILITY	RETAIL_STORE	0	-10		
M103288	Austin	RETAIL_STORE	-43	-43		
M103588	Austin	RETAIL_STORE	18	-31		
M102489	Warehouse	WAREHOUSE	-1	-1		
M102871	MAPLEWOOD	RETAIL_STORE	-101	-101		
DLT_ORD_ITM_SOLR_DOC	Austin	RETAIL_STORE	-10	-10		

11 Transfer Orders Without Inventory Reservation

Business Problem:

When transferring stock between facilities, the system should reserve inventory. If it isn't reserved, the transfer may fail or oversell.

```

SELECT
  oh.ORDER_ID AS TRANSFER_ORDER_ID,
  oisg.FACILITY_ID AS FROM_FACILITY_ID,
  oh.ORIGIN_FACILITY_ID AS TO_FACILITY_ID,
  oi.PRODUCT_ID AS PRODUCT_ID,
  oi.QUANTITY AS REQUESTED_QUANTITY,
  COALESCE(SUM(oisgir.QUANTITY), 0) AS RESERVED_QUANTITY,
  oh.ORDER_DATE AS TRANSFER_DATE,
  oh.STATUS_ID AS STATUS
FROM
  ORDER_HEADER oh
JOIN
  ORDER_ITEM oi
  ON oh.ORDER_ID = oi.ORDER_ID
JOIN
  ORDER_ITEM_SHIP_GROUP oisg
  ON oh.ORDER_ID = oisg.ORDER_ID
LEFT JOIN
  ORDER_ITEM_SHIP_GRP_INV_RES oisgir
  ON oi.ORDER_ID = oisgir.ORDER_ID
  AND oi.ORDER_ITEM_SEQ_ID = oisgir.ORDER_ITEM_SEQ_ID
  AND oisg.SHIP_GROUP_SEQ_ID = oisgir.SHIP_GROUP_SEQ_ID
WHERE
  oh.ORDER_TYPE_ID = 'TRANSFER_ORDER' AND oh.STATUS_ID NOT IN
('ORDER_COMPLETED', 'ORDER_CANCELLED', 'ORDER_REJECTED')
  AND oi.STATUS_ID != 'ITEM_CANCELLED'
GROUP BY
  oh.ORDER_ID,
  oisg.FACILITY_ID,
  oh.ORIGIN_FACILITY_ID,
  oi.PRODUCT_ID,
  oi.QUANTITY,
  oh.ORDER_DATE,
  oh.STATUS_ID
HAVING COALESCE(SUM(oisgir.QUANTITY), 0) < oi.QUANTITY;

```

12 Orders Without Picklist

Business Problem:

A picklist is necessary for warehouse staff to gather items. Orders missing a picklist might be delayed and need attention.

```
SELECT
    oh.ORDER_ID AS ORDER_ID,
    oh.ORDER_DATE AS ORDER_DATE,
    oh.STATUS_ID AS ORDER_STATUS,
    oisg.FACILITY_ID AS FACILITY_ID,
    TIMESTAMPDIFF(HOUR, oh.ORDER_DATE, CURRENT_TIMESTAMP) AS
DURATION_IN_HOURS
FROM
    ORDER_HEADER oh
JOIN
    ORDER_ITEM_SHIP_GROUP oisg
    ON oh.ORDER_ID = oisg.ORDER_ID
LEFT JOIN
    PICKLIST_ITEM pli
    ON oh.ORDER_ID = pli.ORDER_ID
    AND oisg.SHIP_GROUP_SEQ_ID = pli.SHIP_GROUP_SEQ_ID
WHERE
    oh.ORDER_TYPE_ID = 'SALES_ORDER'
    AND oh.STATUS_ID = 'ORDER_APPROVED'
    AND oisg.FACILITY_ID IS NOT NULL
    AND pli.PICKLIST_BIN_ID IS NULL
GROUP BY
    oh.ORDER_ID,
    oh.ORDER_DATE,
    oh.STATUS_ID,
    oisg.FACILITY_ID;
```